UNIT – 2

ASSESSMENT – 3

SOURCE CODE

```
1 import heapq


class AStar:


    def     __init__(self):
self.graph       =       {}
self.heuristic = {}


    # ------------------------
    # Add Edge
    # ------------------------      def
add_edge(self, u, v, cost):


        if      u       not       in       self.graph:
self.graph[u] = []
```

```
        if v not in self.graph:
            self.graph[v] = []

        self.graph[u].append((v,                                cost))
self.graph[v].append((u, cost))

    # ------------------------
    # Print Open List
    # ------------------------        def
print_open(self, open_list):

        print("\nOPEN LIST")        print("-------------
------------------------------")
print("{:<8}{:<8}{:<8}{:<8}".format(
            "Node", "g", "h", "f"))        print("------------------------
--------------------")

        temp = sorted(open_list)

        for item in temp:

            f, g, node, path = item
```

```
            print("{:<8}{:<8}{:<8}{:<8}".format(
                node,
                g,
self.heuristic[node],
                f
            ))

    # ------------------------
    # Print Closed List     # -------
------------------                    def
print_closed(self, closed):

        print("\nCLOSED LIST")

        if        len(closed)        ==        0:
print("Empty")

else:
            for   i   in   closed:
print(i, end=" ")
```

```
print()
    # ------------------------
    # A* Search
    #         ------------------------
def search(self, start, goal):

        open_list = []

        closed = []

        heapq.heappush(
open_list,
            (
self.heuristic[start],
                0,
start,
                [start]
            )
        )

        iteration = 1
```

```
        while open_list:

            f, g, current, path = heapq.heappop(open_list)

            if current in closed:
                continue
             print("\n")            print("=" *
60)            print("ITERATION",
iteration)            print("=" * 60)

            print("\nCurrent Node :", current)

            print("g(n) =", g)             print("h(n)
=", self.heuristic[current])            print("f(n)
=", f)

            closed.append(current)

            if current == goal:

                print("\nGOAL NODE FOUND")
```

```
            print("\nOptimal Path")

            print(" -> ".join(path))

            print("\nTotal Cost =", g)
            return

        for neighbour, cost in self.graph[current]:

            if neighbour not in closed:

                new_g = g + cost

                new_f = new_g + self.heuristic[neighbour]

                heapq.heappush(
open_list,
                    (
new_f,                      new_g,
neighbour,                      path +
[neighbour]
```

```
                )
            )

        self.print_open(open_list)

        self.print_closed(closed)
        if len(open_list) > 0:

            temp = sorted(open_list)

            print("\nSelected Next Node :", temp[0][2])

            print("Reason : Lowest f(n) Value")

        iteration += 1


# ==========================================
# MAIN PROGRAM
# ==========================================

obj = AStar()
```

```
print("\nA* SEARCH ALGORITHM\n")

print("1. Use Faculty Graph") print("2.
Enter Custom Graph")

choice = int(input("\nEnter Choice : "))
if choice == 1:

    obj.add_edge("A", "B", 2)
obj.add_edge("A",   "C",   4)
obj.add_edge("B",   "C",   3)
obj.add_edge("B",   "D",   7)
obj.add_edge("B",   "E",   2)
obj.add_edge("C",   "E",   3)
obj.add_edge("D",   "E",   2)
obj.add_edge("D", "G", 2)

    obj.heuristic["A"] = 7
obj.heuristic["B"] = 6
obj.heuristic["C"] = 4
obj.heuristic["D"] = 3
```

```
obj.heuristic["E"] = 2
obj.heuristic["G"] = 0

else:

    n = int(input("\nEnter Number of Nodes : "))

    nodes = []
    for i in range(n):

        node = input("Node Name : ")

        nodes.append(node)

        obj.graph[node] = []

    e = int(input("\nEnter Number of Edges : "))

    for i in range(e):

        print("\nEdge", i + 1)
```

```
u = input("From : ")

v = input("To : ")

        cost = int(input("Cost : "))

        obj.add_edge(u, v, cost)

    print("\nEnter Heuristic Values\n")
    for node in nodes:

        obj.heuristic[node]              =              int(
input(f"h({node}) : ")
        )

start = input("\nEnter Start Node : ")

goal = input("Enter Goal Node : ")

obj.search(start, goal)
```

2. import math

iteration = 1

def minimax_alpha_beta(left, right):

    global iteration

    alpha = -math.inf beta
    =   math.inf
    print("\n=========
    = MINIMAX WITH
    ALPHA-BETA
    PRUNING
    ==========\n")

    # ---------------- LEFT MIN ----------------

    print("---")
print("Iteration", iteration)   print("Evaluating
LEFT MIN Node")
    print("---")

```
    left_min = math.inf

    for i in range(len(left)):

        print("\nLeaf Node :", left[i])

        left_min = min(left_min, left[i])

        beta = min(beta, left_min)

        print("Alpha =", alpha)       print("Beta
=", beta)         print("Current Minimum =",
left_min)  print("\nLEFT  MIN  returns  :",
left_min)

    alpha = max(alpha, left_min)

    print("\nMAX updates Alpha =", alpha)

    iteration += 1
```

```
    # ---------------- RIGHT MIN ----------------

    print("\n-----------------------------------------------")
print("Iteration", iteration)     print("Evaluating
RIGHT MIN Node")
    print("-----------------------------------------------")

    beta = math.inf

    right_min = math.inf

    pruned = []

    for i in range(len(right)):

        print("\nLeaf Node :", right[i])

        right_min = min(right_min, right[i])

        beta = min(beta, right_min)
```

```
        print("Alpha =", alpha)        print("Beta
=", beta)          print("Current Minimum =",
right_min)

        if beta <= alpha:

            print("\nCondition β ≤ α satisfied")

            print("Remaining nodes are pruned.")

            for j in range(i+1,len(right)):
pruned.append(right[j])

break

    print("\nRIGHT MIN returns :", right_min)

    iteration += 1
    # ---------------- MAX NODE ----------------
```

```
    print("\n-----------------------------------------")
print("Iteration", iteration)     print("Evaluating
ROOT MAX")
    print("-----------------------------------------")

    result = max(left_min,right_min)

    print("\nLeft       Value              :",left_min)
print("Right Value :",right_min)

    print("\nMAX chooses :",result)

    print("\n================       FINAL       RESULT
================")

    if result==left_min:
        print("Best Move : LEFT Subtree")
    else:
        print("Best Move : RIGHT Subtree")

    print("Final Minimax Value :",result)
    if len(pruned)==0:
```

```
        print("Pruned Nodes : None")
    else:
        print("Pruned Nodes :",pruned)


# ---------------- MAIN ----------------

print("MINIMAX WITH ALPHA-BETA PRUNING\n")

print("Enter LEFT MIN Leaf Values\n")

left=[]

for i in range(3):

    value=int(input(f"Leaf {i+1} : "))

    left.append(value)

print("\nEnter RIGHT MIN Leaf Values\n")

right=[]
```

```
for i in range(3):
    value=int(input(f"Leaf {i+1} : "))
    right.append(value)
minimax_alpha_beta(left,right)
```